Accountability Bud - Project History

Accountability Bud - Project History

Date: 2026-05-14 **Author:** Mihail Kenarov

---

1. Assignment Context

This project was built as part of an NLP course assignment. The brief asked students to create a novel agent for solving a particular language task that an LLM cannot do out of the box - meaning some functionality had to be built on top of an LLM rather than relying on its base capabilities alone. The assignment explicitly suggested combining large language models with domain-specific information, external tool integrations, vector databases, and alternative user interfaces.

The response to that brief was to build something genuinely useful rather than a demo. The question asked was: what problem do I actually have that an LLM, properly grounded with real data, could help solve? The answer was daily planning and accountability. An LLM alone cannot know your calendar, remember what you told it last Tuesday, or track whether you have been training consistently. But an agent with the right tools can do all of that - and do it in a way that feels personal and honest rather than generic.

The formal design challenge statement for this project is:

> *Design a conversational life coach web agent to enable busy students in managing a demanding and irregular daily schedule to plan their day realistically and stay accountable to their personal goals, with consistent improvement in training frequency, time management, and self-awareness over multiple weeks.*

---

2. The Project: Accountability Bud

Accountability Bud is a conversational life coach web app built for a busy student with an irregular schedule, training commitments, calorie goals, and a need for a private accountability partner. It is not a generic chatbot - it knows your real calendar, remembers facts about you across sessions, tracks your habits over time, and uses all of that context to give grounded coaching instead of vague motivational advice.

The app is designed around three core functionalities:

- **Planning** - the agent reads your Google Calendar, pulls relevant facts it has learned about you, checks your habit streaks, and generates a realistic time-blocked plan for the day
- **Check-in** - when life changes (you skipped the gym, work ended early, plans shifted), the agent retrieves the current plan and helps you adjust without judgment
- **Reflection** - the user reports what they did, the agent extracts habit completions from free text, logs them to a database, updates streaks, and saves a session summary for future reference

The brand personality is deliberate: calm, direct, and honest. No confetti, no badges, no gamification. Streaks are shown as plain numbers. Plans are presented as honest time blocks. The design borrows from the ledger book and the personal planner - a trusted daily tool, not a productivity app trying to motivate you with visual noise.

---

3. Core Functionalities

Planning

The primary use case. The user asks the agent to plan the day. The agent reads the current Google Calendar events, retrieves relevant facts it has learned about the user from past conversations (training schedule, goals, recurring constraints), checks the current habit streaks, and generates a realistic time-blocked plan. The plan is saved to the database so it can be referenced later. This is where the grounding matters most - the agent cannot suggest a gym session at 10:00 if there is already a lecture on the calendar at that time.

Check-In

Life rarely goes exactly to plan. The check-in handles the moments when things change - the user skipped the gym, work ended early, energy is low, or something unexpected came up. The agent retrieves the current plan, takes in what the user reports, and helps adjust the rest of the day without judgment. The goal is to keep the user moving forward with a realistic revised plan rather than treating a deviation as a failure.

Reflection

The user reports back on their day - what they did, what they skipped, how it went. The agent extracts habit completions from the user's free-text response (did they train, stay on calories, read), logs each habit to the database, updates the streaks, and saves a summary of the session to long-term memory. The response includes what went well, an honest acknowledgment of what slipped, and one concrete suggestion for the next day. This session summary is retrieved the next time the user asks for a plan, closing the loop.

---

4. Technology Choices

Every technology in this project was chosen for a specific reason. Several were recommended directly in the NLP assignment brief; others were chosen after weighing alternatives.

LangChain

The assignment brief recommended LangChain as one of the primary frameworks for building NLP agents, and it was one of the most widely adopted agent frameworks in the community at the time of building. Prior experience was with Semantic Kernel - a comparable agentic framework - but this project was a deliberate opportunity to try something new and learn a tool with broader community usage. LangChain's `create_tool_calling_agent` and `AgentExecutor` made it straightforward to wire together LLM calls, tool definitions, and conversation history in a single composable pipeline.

OpenRouter

OpenRouter provides access to capable cloud-hosted language models through a single unified API. It was chosen as the LLM provider for this project (more on the journey to this decision in the iterations section below).

Google Calendar API + OAuth

Real schedule data was a hard requirement - if the agent generates a plan without knowing what is already on the calendar, it is just guessing. The Google Calendar API provides exactly that: live access to the user's actual events. OAuth was used for authentication because it is the standard, secure flow for user-delegated access to Google services, and it avoids storing credentials directly in the application.

FastMCP / MCP Server

The Google Calendar integration is exposed through a local MCP (Model Context Protocol) server built with FastMCP, rather than being implemented as a hardcoded LangChain tool. This separation means the calendar integration is a reusable, composable external service that could be consumed by other agents or applications in the future. LangChain connects to it via `langchain-mcp-adapters`, which converts MCP tools into native LangChain tool definitions automatically.

ChromaDB

ChromaDB is used as the semantic memory layer - storing facts learned about the user and summaries of past conversations, retrieved by meaning rather than exact match. When the agent needs to know "what do I know about this user's training schedule?", it performs a vector similarity search across all stored facts and returns the most relevant ones. This kind of retrieval is not possible with a traditional relational database. ChromaDB was chosen for its simple local setup and persistent storage without requiring an external service.

SQLite

SQLite handles all structured data: daily plans, habit logs, and habit streaks. These require precise queries - exact dates, numeric streak counts, completion rates - not semantic similarity. Having two separate storage backends (ChromaDB for meaning-based retrieval, SQLite for exact structured queries) was a deliberate architectural decision. Each backend is used for the type of data it is actually good at.

Streamlit

Streamlit was chosen for the frontend because it is fast to build with, mobile-accessible out of the box, and well-suited to a daily-use interface that needs a chat surface plus live context panels. It allowed the full UI to be built and iterated on quickly without requiring a separate frontend stack.

---

5. Iteration One: The First Working Version

The project started with a clear architecture in mind: a LangChain agent wrapping a local Ollama LLM, all backend tools built and tested, and a Streamlit frontend wiring it all together. The initial model choice was `ministral-3b` running locally via Ollama - free, private, and requiring no API key.

The core backend was built and fully tested first:

- **Google Calendar MCP server** - list, create, update, and delete events via the Google Calendar API, exposed as MCP tools
- **ChromaDB tools** - store and retrieve user facts with confidence filtering and deduplication; save and retrieve conversation session summaries
- **Fact extractor** - a background module that runs after every user message, extracts structured facts about the user using the LLM, and stores them silently in ChromaDB
- **SQLite habit tracker** - daily habit logs, plan storage, and streak aggregation across `habit_logs`, `daily_plans`, and `habit_streaks` tables

The test suite covered all of these components with 20 passing tests before the Streamlit frontend was wired in.

The first iteration was a genuine success: Ollama connected to the Streamlit frontend, all tools wired, end-to-end flow working. The first live demo ran on this stack. However, a hardware constraint emerged: a separate local speech-to-text model was running alongside Ollama, and the two together consumed more VRAM than the

machine could comfortably handle. Both models slowed significantly under the combined load, making the app frustrating to use in practice.

---

6. Iteration Two: Switching the LLM Provider

The VRAM constraint had a straightforward solution: move the LLM to the cloud and free up local resources for the speech-to-text model. OpenRouter was integrated as the new LLM provider, replacing the direct Ollama connection.

To keep the codebase flexible, a new `agent/llm.py` module was introduced as a unified model builder. It reads configuration from environment variables and returns either an OpenRouter-backed or Ollama-backed chat model depending on the setup - meaning the original Ollama path was not removed, just made optional. A better-resourced machine could switch back to fully local operation by changing a single environment variable.

The application remained functionally identical to iteration one. The only change was where the LLM inference happened.

---

7. Iteration Three: UI Polish and Data Model Improvements

With the LLM provider stable, the third iteration focused on making the app genuinely pleasant and useful to interact with daily.

Google Calendar timezone normalization - the calendar integration was enhanced with proper timezone handling, ensuring events from different timezones were normalized consistently before being displayed or reasoned about by the agent.

Visual day-timeline - the calendar panel in the UI was rebuilt as a 24-hour day grid with event blocks positioned at their actual times and a current-time indicator. This made the day's schedule immediately readable at a glance, rather than being a plain text list.

Daily todos as structured elements - the daily plan storage was refactored from a single plain text string into individual structured todo items. Each todo is a discrete record in SQLite rather than a block of text. The planning agent was updated via prompt changes to save agreed todos as structured items after a planning conversation, and the dashboard UI was updated to render them as a proper checklist rather than raw text.

Visual design system - a full design system was documented for the app under the name "The Quiet Ledger." The palette is a single sage-green note played across an almost-white ground, with one accent color appearing only on interactive elements and confirmed states. Typography uses the system UI font at variable weights - no web fonts, no display face. The design explicitly rejects gamification, dashboard clutter, and generic chatbot aesthetics.

---

8. Validation Strategy

The NLP assignment asks how you ensure an app is trustworthy given the probabilistic nature of LLMs. Several deliberate decisions address this:

Calendar data as ground truth - plans are always built on top of real Google Calendar events retrieved live from the API. The LLM cannot schedule something during a blocked time slot because it always sees the actual calendar before generating a plan.

Structured output for habit logging - the LLM extracts intent from the user's free-text evening report ("did the user train today?"), but a tool writes the actual database record. The LLM never writes a streak number or a completion status directly - it extracts meaning, and the tool handles the write.

Confidence threshold for fact storage - user facts extracted from conversation are only stored in ChromaDB if the LLM assigns them a confidence above a minimum threshold. Low-confidence or speculative extractions are discarded rather than allowed to pollute the memory.

Deduplication - before storing a new fact, ChromaDB is queried for semantically similar existing facts. If a near-duplicate already exists, the new one is skipped. This prevents contradictory or repeated entries from accumulating over time.

Confirmation before calendar writes - the agent always proposes a calendar change and waits for explicit user confirmation before calling create, update, or delete on Google Calendar. The user is always in control of what gets written.

Fixed, version-controlled system prompt - the agent's role, tone, and behavioral constraints are defined in a single `prompts.py` file that is committed to the repository. The LLM's behavior does not drift between sessions.

---

9. Possible Future Additions

Several natural next steps were considered but not yet implemented:

- **Voice input** - a local or cloud-based speech-to-text integration for hands-free morning check-ins. This is what originally caused the VRAM issue in iteration one, but it remains a compelling feature worth revisiting with a cloud transcription API or better hardware.
- **Mobile app** - a proper native or progressive web app version for quick mid-day check-ins without needing to open a laptop.
- **Proactive notifications** - the agent nudging the user at set times ("it's 18:00, did you train today?") rather than only responding when the user initiates a conversation.
- **Multi-user support** - generalizing the architecture beyond a single user's calendar and habits to support multiple students, each with their own isolated data store.